Infrastructure / SRE Intern Assignment Submission
Candidate Name: Sheersh Sinha
GitHub Repository: https://github.com/Sheersh123/Incident-Management-System.git

1. Executive Summary

This document outlines the implementation of a Mission-Critical Incident Management System (IMS). The system is designed to handle high-throughput signal ingestion (10,000 signals/sec) while maintaining high availability and transactional integrity. The architecture leverages a distributed stack including FastAPI, Apache Kafka, PostgreSQL, MongoDB, and Redis.

2. Architecture Overview

The system follows a decoupled, asynchronous processing model to ensure resilience under load.

Ingestion Layer: FastAPI producers publish signals to Kafka topics.
Messaging Layer: Apache Kafka acts as a high-performance buffer, providing backpressure management.
Processing Layer: Dedicated workers consume from Kafka, implementing debouncing logic and state management.
Storage Strategy:
PostgreSQL: Source of Truth for transactional incident and RCA data.
MongoDB: NoSQL Data Lake for high-volume raw signal audit logs.
Redis: Hot-path dashboard state and rate-limiting.

GitHub Link for Full Codebase:

https://github.com/Sheersh123/Incident-Management-System.git

3. High-Performance & Non-Functional Features

As per the requirement for performance and security:

Backpressure Handling: Kafka allows the system to survive bursts by queuing messages when the database is under load.
Rate Limiting: Implemented a Redis-backed sliding window rate limiter on the ingestion API to prevent DDoS and cascading failures.
Atomic Debouncing: Uses Redis atomic operations to deduplicate 100+ identical signals within a 10-second window into a single incident.
Security Layer: Pydantic-based schema validation at the edge to prevent malformed data injection.

4. Design Patterns Used

Strategy Pattern (Alerting): Swappable alerting logic (P0/P1/P2) based on component failure severity.
State Pattern (Lifecycle): Strict control over incident transitions (Open → Investigating → Resolved → Closed), enforcing a mandatory RCA submission before closure.

5. Running the Application

The application is fully containerized for a "one-click" setup.

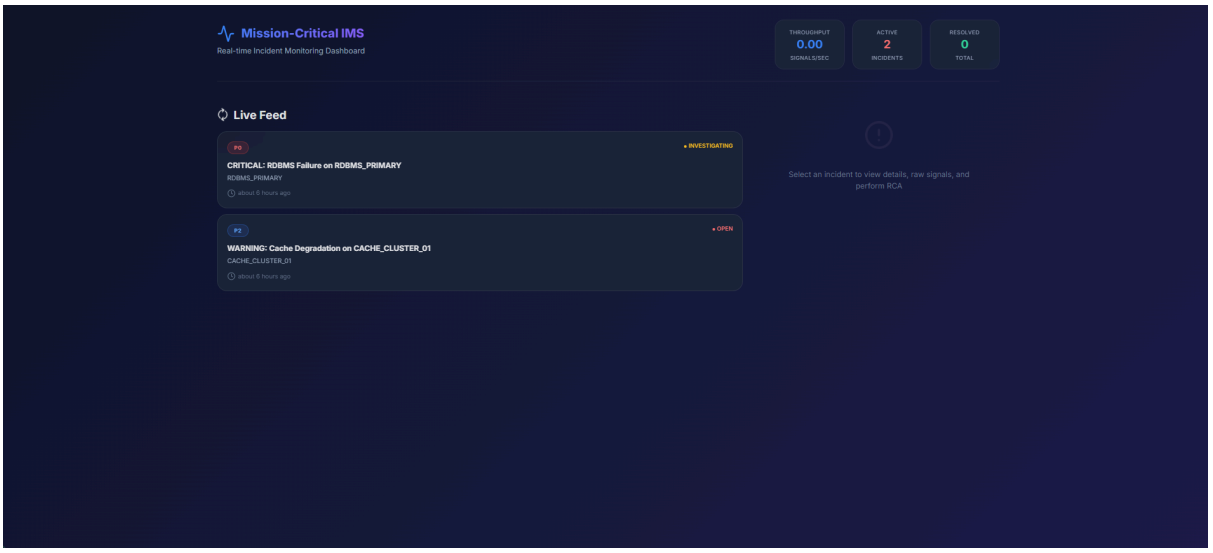
Navigate to the /backend directory.

Run docker-compose up --build -d.
Access the Dashboard at http://localhost:5173.
Access the API Docs at http://localhost:8000/docs.

6. Failure Scenario Simulation

I have included a script to simulate a complex failure event across the stack:

Script Location: backend/scripts/seed_data.py

Scenario: Simulates an RDBMS Primary failure followed by a cascading MCP host failure.

7. Dashboard Preview